

贵州大学
GUIZHOU UNIVERSITY

实践（实验）报告

课程名称： Java Web 后端开发技术

学　　院： 示范性软件工程学院

授课教师： 丁玺润

专业班级： 软件工程 2403

学　　号： 2400770225

姓　　名： 皮宸西

小组成员： 皮宸西

2026年10月22日

实 验 名 称：	实验三：MyBatis-Plus 持久层开发

一、实验目的：

1. 理解 MyBatis-Plus 的定位与优势，掌握其在 Spring Boot 中的集成配置。

2. 掌握实体类与数据表的映射（@TableName、@TableId、@TableField）。

3. 掌握 Mapper 接口继承 BaseMapper 后的 CRUD 能力，以及注解 SQL 的编写。

二、实验环境：

硬件环境：PC 计算机（内存 8GB 及以上）。

软件环境：Windows 11、JDK 17.0.20.1（Temurin）、Maven 3.9.16、IntelliJ IDEA、Spring Boot 3.3.5、MyBatis-Plus 3.5.9、MySQL 8.0.29、Apifox。

网络环境：可访问 Maven 中央仓库（已配置阿里云镜像）。

三、实验内容：

1. 在 CTTS 项目中集成 MyBatis-Plus，完成依赖与配置。

2. 创建 sys_user、training_project 数据表及对应实体类、Mapper 接口。

3. 实现用户、实训项目的增删改查接口，并用 Apifox 验证。

四、实验步骤（包含运行结果截图）：

（1）梳理工程结构，明确实验三的任务边界

实验一完成了 CTTS 工程骨架（com.gzu:ctts，Spring Boot 3.3.5，JDK 17，Maven 构建），实验二补齐了 Controller 层的接口，但数据保存在 Service 层的一个 CopyOnWriteArrayList 中——进程一重启数据全部还原，持久层实际上是空的。实验三的任务就是把这套“内存数据库”换成真正的 MySQL：引入 MyBatis-Plus，为 sys_user 与 training_project 两张表建立实体类与 Mapper 接口，把 Service 的读写改造为数据库访问，并把接口从“只能查和增”补全为完整的增删改查。

实验三新增与改造的类及其职责见表 3.1，改造后的工程目录结构如图 3.1 所示。

表 3.1 实验三新增与改造的类及职责

所在包	类 / 文件	职责
com.gzu.ctts.entity	User.java（新增）	映射 sys_user 表，演示 @TableName 、 @TableId 、 @TableField、@TableLogic
com.gzu.ctts.entity	Project.java（改造）	映射 training_project 表，属性 name 对应列 project_name
com.gzu.ctts.mapper	UserMapper.java（新增）	继承 BaseMapper<User>，并补充 @Select/@Update 注解 SQL
com.gzu.ctts.mapper	ProjectMapper.java（新增）	继承 BaseMapper<Project>，用 @Results 显式声明结果映射
com.gzu.ctts.config	MybatisPlusConfig.java（新增）	注册分页插件 PaginationInnerInterceptor
com.gzu.ctts.config	MyMetaObjectHandler.java （新增）	创建时间、更新时间的自动填充
com.gzu.ctts.service	UserService / UserServiceImpl（新增）	用户业务逻辑，数据来源改为 UserMapper
com.gzu.ctts.service	ProjectService / ProjectServiceImpl	由内存 List 改为调用

	（改造）	ProjectMapper
com.gzu.ctts.controller	UserController.java（新增）	用户模块 8 个 REST 接口
com.gzu.ctts.controller	ProjectController.java（改造）	实训项目模块 8 个 REST 接口
com.gzu.ctts.dto	UserForm.java（新增）	用户请求参数对象及其校验规则
sql/ctts.sql	建库脚本（扩充）	新增 sys_user、training_project 两张表及初始化数据

图 3.1 CTTS 工程目录结构（实验三新增持久层）

```
...\实验3\ctts> tree /F src

src
  main
    java
      com
        gzu
          ctts
            CttsApplication.java
            config
              MyMetaObjectHandler.java
              MybatisPlusConfig.java
              OpenApiConfig.java
            controller
              ProjectController.java
              UserController.java
            dto
              PageResult.java
              ProjectForm.java
              Result.java
              UserForm.java
            entity
              Project.java
              User.java
            exception
              BizException.java
              GlobalExceptionHandler.java
            mapper
              ProjectMapper.java
              UserMapper.java
            service
              ProjectService.java
              UserService.java
              impl
                ProjectServiceImpl.java
                UserServiceImpl.java
    resources
      application.yml
  test
    java
      com
        gzu
          ctts
            CttsApplicationTests.java

说明：实验三在实验二基础上新增 mapper 包（MyBatis-Plus Mapper 接口），
并扩充 entity（User）、config（MybatisPlusConfig、MyMetaObjectHandler）、
dto（UserForm）、service（UserService）、controller（UserController）。
```

图 3.1　CTTS 工程目录结构（实验三新增持久层）

（2）在 pom.xml 中引入 MyBatis-Plus 与分页插件依赖

MyBatis-Plus 不在 Spring Boot 官方父工程的依赖管理范围内，必须先在 properties 中显式声明版本号，再在 dependencies 中引用，否则 Maven 会因找不到版本而构建失败。此外还有一个容易踩的坑：自 MyBatis-Plus 3.5.9 起，分页插件依赖的 SQL 解析器被拆分到独立的 mybatis-plus-jsqlparser 模块，若只引入 starter 而不引入该模块，配置分页插件时会抛 NoClassDefFoundError。本次实验在构建阶段就遇到了这个问题，补充依赖后构建通过。依赖配置如图 3.2 所示，各依赖的作用见表 3.2。

图 3.2 pom.xml MyBatis-Plus 依赖配置（第 38–54 行）

```
38
39          <!-- ① 持久层：MyBatis-Plus（Spring Boot 3 专用 starter） -->
40          <dependency>
41              <groupId>com.baomidou</groupId>
42              <artifactId>mybatis-plus-spring-boot3-starter</artifactId>
43              <version>${mybatis-plus.version}</version>
44          </dependency>
45
46          <!-- ①-1 分页插件所需的 SQL 解析器。
47               自 MyBatis-Plus 3.5.9 起，PaginationInnerInterceptor（分页插件）
48               已被拆分到独立的 mybatis-plus-jsqlparser 模块，
49               若不单独引入，配置分页插件时会抛 ClassNotFoundException。 -->
50          <dependency>
51              <groupId>com.baomidou</groupId>
52              <artifactId>mybatis-plus-jsqlparser</artifactId>
53              <version>${mybatis-plus.version}</version>
54          </dependency>
```

图 3.2 pom.xml MyBatis-Plus 依赖配置

表 3.2 pom.xml 新增依赖及作用

依赖坐标	版本	作用
spring-boot-starter-web	3.3.5（父工程仲裁）	内嵌 Tomcat + Spring MVC + Jackson
mybatis-plus-spring-boot3-starter	3.5.9	MyBatis-Plus 的 Spring Boot 3 专用起步依赖，提供 BaseMapper、条件构造器与自动配置

mybatis-plus-jsqlparser	3.5.9	分页插件所需的 SQL 解析器，3.5.9 起必须单独引入
mysql-connector-j	父工程仲裁	MySQL 8 的 JDBC 驱动（runtime 作用域）
spring-boot-starter-validation	3.3.5	提供 @NotBlank、@Size 等 JSR-380 校验注解
springdoc-openapi-starter-webmvc-ui	2.6.0	生成 /v3/api-docs，供 Apifox 一键导入接口

（3）完善 application.yml 中的 MyBatis-Plus 配置

配置分为三部分：configuration 控制 MyBatis 原生行为，global-config 控制 MyBatis-Plus 的全局策略，type-aliases-package 为实体类注册别名。其中把 log-impl 设为 StdOutImpl 是本次实验的关键调试手段——打开后控制台会打印每条真正执行的 SQL 及其参数，后面“MyBatis-Plus 到底生成了什么 SQL”的证据正来源于此。配置内容如图 3.3 所示，关键配置项的含义见表 3.3。

图 3.3 application.yml MyBatis-Plus 配置（第 26–42 行）

```yaml
# MyBatis-Plus 配置
mybatis-plus:
  # 实体类所在包：配置后 XML/注解中的 resultType 可直接写类名（如 User）
  type-aliases-package: com.gzu.ctts.entity
  configuration:
    # 开启驼峰命名自动映射：数据库 real_name -> 实体 realName、create_time -> createTime
    map-underscore-to-camel-case: true
    # 开发期把 SQL 打印到控制台，便于调试（实验报告中「自动生成的 SQL」即来源于此）
    log-impl: org.apache.ibatis.logging.stdout.StdOutImpl
  global-config:
    banner: false
    db-config:
      # 主键策略：数据库自增，等价于实体类 @TableId(type = IdType.AUTO)
      id-type: auto
      # 逻辑删除：@TableLogic 标注的字段，delete 语句会被自动改写成 update
      logic-delete-value: 1
      logic-not-delete-value: 0
```

图 3.3 application.yml MyBatis-Plus 配置

表 3.3 application.yml 中 MyBatis-Plus 关键配置项

配置项	取值	作用
type-aliases-package	com.gzu.ctts.entity	为实体类注册类型别名，注解 SQL 中可直接写类名
map-underscore-to-camel-case	true	下划线列名自动映射为驼峰属性，如 real_name → realName
log-impl	StdOutImpl	把执行的 SQL 与参数打印到控制台，便于调试与取证
id-type	auto	全局主键策略为数据库自增，等价于 @TableId(type = IdType.AUTO)
logic-delete-value / logic-not-delete-value	1 / 0	逻辑删除的“已删除/未删除”取值约定

（4）启动 MySQL 并执行建库脚本，创建两张实验表

实验环境中的 MySQL 8.0.29 为免安装版，需先用 mysqld --defaults-file 启动服务，确认 3306 端口进入 LISTENING 状态后，再执行工程提供的 sql/ctts.sql 完成建库建表。脚本除保留实验一的 tb_task、tb_submission 外，新增实验三要使用的两张表：sys_user（系统用户表，7 条初始数据）与 training_project（实训项目表，8 条初始数据）。

sys_user 的字段设计刻意覆盖了几种典型的映射场景：real_name、create_time 等下划线列名用于验证驼峰映射；password 用于演示 @TableField(select = false)；deleted 用于演示 @TableLogic 逻辑删除。建表语句如图 3.4 所示，字段

说明见表 3.4。

图 3.4 sys_user 建表语句（第 50–74 行）

```
50  -- 实验三新增：用户表（对应实体类 com.gzu.ctts.entity.User）
51  --
52  -- 字段命名刻意覆盖实体映射的几种典型场景：
53  --   * real_name / create_time 等下划线列名 —— 演示「下划线转驼峰」与 @TableField
54  --   * password                —— 演示 @TableField(select = false) 查询时不返回
55  --   * deleted                 —— 演示 @TableLogic 逻辑删除
56  --   * create_time / update_time —— 演示 @TableField(fill = ...) 自动填充
57  -- ============================================================
58  DROP TABLE IF EXISTS `sys_user`;
59  CREATE TABLE `sys_user` (
60      `id`          BIGINT       NOT NULL AUTO_INCREMENT            COMMENT '用户主键 ID',
61      `username`    VARCHAR(50)  NOT NULL                           COMMENT '登录账号，唯一',
62      `password`    VARCHAR(64)  NOT NULL                           COMMENT '密码摘要（本实验存 32 位 MD5）',
63      `real_name`   VARCHAR(50)  DEFAULT NULL                       COMMENT '真实姓名',
64      `role`        VARCHAR(20)  NOT NULL DEFAULT 'STUDENT'         COMMENT '角色：ADMIN-管理员 TEACHER-教师 STU
65      `email`       VARCHAR(100) DEFAULT NULL                       COMMENT '电子邮箱',
66      `phone`       VARCHAR(20)  DEFAULT NULL                       COMMENT '手机号',
67      `status`      TINYINT      NOT NULL DEFAULT 1                 COMMENT '状态：1-启用 0-禁用',
68      `deleted`     TINYINT      NOT NULL DEFAULT 0                 COMMENT '逻辑删除标记：0-未删除 1-已删除',
69      `create_time` DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP  COMMENT '创建时间',
70      `update_time` DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间
71      PRIMARY KEY (`id`),
72      UNIQUE KEY `uk_username` (`username`),
73      KEY `idx_role_status` (`role`, `status`)
74  ) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COMMENT = '系统用户表';
```

图 3.4　sys_user 建表语句

表 3.4 sys_user 表字段说明

字段名	类型	约束 / 默认值	说明	对应实体属性
id	BIGINT	PK, AUTO_INCREMENT	用户主键	id（@TableId）
username	VARCHAR(50)	NOT NULL, UNIQUE	登录账号，唯一	username
password	VARCHAR(64)	NOT NULL	密码摘要（MD5）	password（select=false）
real_name	VARCHAR(50)	NULL	真实姓名	realName（@TableField）
role	VARCHAR(20)	NOT NULL, DEFAULT 'STUDENT'	角色：ADMIN/TEACHER/STUDENT	role
email	VARCHAR(100)	NULL	电子邮箱	email
phone	VARCHAR(20)	NULL	手机号	phone
status	TINYINT	NOT NULL, DEFAULT 1	状态：1-启用 0-禁用	status
deleted	TINYINT	NOT NULL,	逻辑删除标记	deleted

		DEFAULT 0		（@TableLogic）
create_time	DATETIME	DEFAULT CURRENT_TIMESTAMP	创建时间	createTime （自动填充）
update_time	DATETIME	ON UPDATE CURRENT_TIMESTAMP	更新时间	updateTime （自动填充）

training_project 表中列名 project_name 与实体属性 name 不同名，是本次实验专门设置的“不一致场景”，用于验证 @TableField("project_name") 的必要性。

建表语句如图 3.5 所示，字段说明见表 3.5。

图 3.5 training_project 建表语句（第 77–95 行）

```
77  -- 实验三新增：实训项目表（对应实体类 com.gzu.ctts.entity.Project）
78  -- 列名 project_name 与实体属性 name 不同名，
79  -- 用于演示 @TableField("project_name") 的显式列名映射
80  -- ----------------------------------------------------------
81  DROP TABLE IF EXISTS `training_project`;
82  CREATE TABLE `training_project` (
83      `id`           BIGINT       NOT NULL AUTO_INCREMENT           COMMENT '项目主键 ID',
84      `project_name` VARCHAR(100) NOT NULL                          COMMENT '项目名称',
85      `category`     VARCHAR(50)  DEFAULT NULL                      COMMENT '项目类别',
86      `teacher`      VARCHAR(50)  DEFAULT NULL                      COMMENT '指导教师',
87      `deadline`     DATE         DEFAULT NULL                      COMMENT '截止日期',
88      `status`       TINYINT      NOT NULL DEFAULT 0                COMMENT '状态：0-未开始 1-进行中 2-已完成',
89      `description`  VARCHAR(500) DEFAULT NULL                      COMMENT '项目描述',
90      `create_time`  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
91      `update_time`  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间
92      PRIMARY KEY (`id`),
93      KEY `idx_status_deadline` (`status`, `deadline`),
94      KEY `idx_category` (`category`)
95  ) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COMMENT = '实训项目表';
```

图 3.5 training_project 建表语句

表 3.5 training_project 表字段说明

字段名	类型	约束 / 默认值	说明	对应实体属性
id	BIGINT	PK, AUTO_INCREMENT	项目主键	id（@TableId）
project_name	VARCHAR(100)	NOT NULL	项目名称	name（@TableField）
category	VARCHAR(50)	NULL	项目类别	category
teacher	VARCHAR(50)	NULL	指导教师	teacher

deadline	DATE	NULL	截止日期	deadline（LocalDate）
status	TINYINT	NOT NULL, DEFAULT 0	状态：0/1/2	status
description	VARCHAR(500)	NULL	项目描述	description
create_time	DATETIME	DEFAULT CURRENT_TIMESTAMP	创建时间	createTime（自动填充）
update_time	DATETIME	ON UPDATE CURRENT_TIMESTAMP	更新时间	updateTime（自动填充）

（5）编写实体类，完成 Java 对象与数据表的映射

实体类是 MyBatis-Plus 全部能力的起点：BaseMapper 之所以能“不写 SQL 就能增删改查”，正是因为它在运行期读取实体上的映射注解来推导表名、列名与主键策略。

本次实验用到的四个注解及其含义见表 3.6，两个实体类的源码分别如图 3.6、图 3.7 所示。

表 3.6 实体映射注解说明

注解	本实验中的用法	作用
@TableName	@TableName("sys_user")	声明实体对应的表名；类名与表名不一致时必写
@TableId	@TableId(value = "id", type = IdType.AUTO)	声明主键字段与主键生成策略，AUTO 表示数据库自增，插入后主键回填到实体
@TableField	value 指定列名；select = false 查询不返回；fill 指定自动填充时机	声明普通字段与列的映射关系，是 @TableName/@TableId 之外最常用的注解
@TableLogic	@TableLogic 标注 deleted 字段	声明逻辑删除字段，删除操作被改写为 UPDATE，查询自动追加未删除条件

图 3.6 User.java 实体映射注解（第 42–103 行）

```java
@Data
@TableName("sys_user")
@Schema(description = "系统用户")
public class User implements Serializable {

    /** 用户主键 ID：数据库自增，插入后自动回填 */
    @TableId(value = "id", type = IdType.AUTO)
    @Schema(description = "用户主键 ID", example = "5")
    private Long id;

    /** 登录账号，唯一索引列 */
    @Schema(description = "登录账号（唯一）", example = "2024001")
    private String username;

    /**
     * 密码摘要（32 位 MD5）。
     *
     * <p>{@code select = false} 使该列不参与查询，因此接口响应中 password 恒为 null，
     * 避免密码摘要随用户列表泄露。
     */
    @TableField(value = "password", select = false)
    @Schema(description = "密码摘要，查询接口不返回（恒为 null）", example = "null")
    private String password;

    /** 真实姓名：列名 real_name，演示列名与属性名的映射 */
    @TableField("real_name")
    @Schema(description = "真实姓名", example = "皮宸西")
    private String realName;

    /** 角色：ADMIN-管理员 TEACHER-教师 STUDENT-学生 */
    @Schema(description = "角色：ADMIN-管理员，TEACHER-教师，STUDENT-学生",
            example = "STUDENT", allowableValues = {"ADMIN", "TEACHER", "STUDENT"})
    private String role;

    /** 电子邮箱 */
    @Schema(description = "电子邮箱", example = "2400770225@qq.com")
    private String email;

    /** 手机号 */
    @Schema(description = "手机号", example = "13900000005")
    private String phone;

    /** 状态：1-启用 0-禁用 */
    @Schema(description = "状态：1-启用，0-禁用", example = "1", allowableValues = {"0", "1"})
    private Integer status;

    /** 逻辑删除标记：0-未删除 1-已删除，由 @TableLogic 自动维护 */
    @TableLogic
    @Schema(description = "逻辑删除标记：0-未删除，1-已删除", example = "0")
    private Integer deleted;

    /** 创建时间：插入时由 MetaObjectHandler 自动填充 */
    @TableField(value = "create_time", fill = FieldFill.INSERT)
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss")
    @Schema(description = "创建时间，插入时自动填充", example = "2026-10-22 19:30:00")
    private LocalDateTime createTime;

    /** 更新时间：插入与更新时由 MetaObjectHandler 自动填充 */
    @TableField(value = "update_time", fill = FieldFill.INSERT_UPDATE)
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss")
    @Schema(description = "更新时间，插入与更新时自动填充", example = "2026-10-22 19:30:00")
    private LocalDateTime updateTime;
```

图 3.6　User.java 实体映射注解

图 3.7 Project.java 实体映射注解（第 32–87 行）

```java
@Data
@TableName("training_project")
@Schema(description = "实训项目")
public class Project implements Serializable {

    /** 项目主键 ID：数据库自增 */
    @TableId(value = "id", type = IdType.AUTO)
    @Schema(description = "项目主键 ID", example = "3")
    private Long id;

    /** 项目名称：属性名 name 与列名 project_name 不同，需显式映射 */
    @TableField("project_name")
    @Schema(description = "项目名称", example = "实训成果统计报表")
    private String name;

    /** 项目类别，如「Web 后端」「数据分析」 */
    @Schema(description = "项目类别", example = "数据分析")
    private String category;

    /** 指导教师 */
    @Schema(description = "指导教师", example = "李敏")
    private String teacher;

    /**
     * 截止日期，格式 yyyy-MM-dd。
     *
     * <p>数据库中该列类型为 DATE，这里用 {@link LocalDate} 承接；
     * MyBatis 3.4.5 起内置了 JSR-310 类型处理器，无需额外配置即可完成
     * {@code java.sql.Date} 与 {@code LocalDate} 的互转。
     */
    @JsonFormat(pattern = "yyyy-MM-dd")
    @Schema(description = "截止日期，格式 yyyy-MM-dd", example = "2026-11-30")
    private LocalDate deadline;

    /** 状态：0-未开始 1-进行中 2-已完成 */
    @Schema(description = "状态：0-未开始，1-进行中，2-已完成", example = "1", allowableValues = {"0", "1", "2"})
    private Integer status;

    /** 项目描述 */
    @Schema(description = "项目描述", example = "按班级、项目维度统计实训提交率与平均得分。")
    private String description;

    /** 创建时间：插入时自动填充 */
    @TableField(value = "create_time", fill = FieldFill.INSERT)
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss")
    @Schema(description = "创建时间，插入时自动填充", example = "2026-10-22 19:30:00")
    private LocalDateTime createTime;

    /** 更新时间：插入与更新时自动填充 */
    @TableField(value = "update_time", fill = FieldFill.INSERT_UPDATE)
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss")
    @Schema(description = "更新时间，插入与更新时自动填充", example = "2026-10-22 19:30:00")
    private LocalDateTime updateTime;
}
```

图 3.7　Project.java 实体映射注解

（6）编写 Mapper 接口，继承 BaseMapper 获得 CRUD 能力

Mapper 接口只有方法声明、没有实现类：MyBatis 启动时扫描 `@MapperScan` 指定的包，为每个接口生成 JDK 动态代理对象，方法调用被代理拦截后，依据方法签名（以

及注解）装配 SQL 并交给 SqlSession 执行。继承 `BaseMapper<T>` 后，单表增删改查全部无需手写 SQL，常用的方法见表 3.7；当遇到聚合统计、多表关联等 `BaseMapper` 覆盖不到的场景，则用 `@Select`、`@Update` 等注解 SQL 补齐。两个 Mapper 的源码如图 3.8、图 3.9 所示。

表 3.7 BaseMapper 提供的常用方法（本实验实际调用）

方法	对应 SQL	本实验调用位置
`insert(entity)`	INSERT INTO … VALUES (…)	新增用户 / 新增实训项目
`deleteById(id)`	DELETE … / UPDATE … SET deleted=1（逻辑删除时）	删除用户、删除实训项目
`updateById(entity)`	UPDATE … SET … WHERE id = ?	修改用户、修改实训项目
`selectById(id)`	SELECT … WHERE id = ?	用户/项目详情、Service 层存在性校验
`selectCount(wrapper)`	SELECT COUNT(*) … WHERE …	新增用户时的账号唯一性校验
`selectPage(page, wrapper)`	SELECT COUNT(*) … 与 SELECT … LIMIT ?, ?	两个模块的分页查询
`selectList(wrapper)`	SELECT … WHERE …	用户列表、项目列表查询

图 3.8 UserMapper.java（继承 BaseMapper 并补充注解 SQL）（第 38–85 行）

```java
public interface UserMapper extends BaseMapper<User> {

    /**
     * 按角色查询用户列表（注解 SQL 示例）。
     *
     * <p>{@code #{role}} 是 MyBatis 的占位符，最终会被编译成 JDBC 的 {@code ?}
     * 并通过 PreparedStatement 预编译传参，可以防止 SQL 注入；
     * 与之相对的 {@code ${}} 是字符串拼接，仅在动态表名/列名时使用，存在注入风险。
     *
     * <p>{@code @Param("role")} 给参数命名，供 {@code #{role}} 引用；
     * 单参数且为简单类型时可省略，多参数时必须写。
     */
    @Select("""
            SELECT id, username, real_name, role, email, phone, status, deleted, create_time, update_time
            FROM sys_user
            WHERE role = #{role} AND deleted = 0
            ORDER BY id
            """)
    List<User> selectByRole(@Param("role") String role);

    /**
     * 修改用户启用状态（注解 SQL 示例）。
     *
     * <p>返回值为受影响的行数：0 表示 id 不存在或已被逻辑删除，
     * Service 层据此判断操作是否成功。
     */
    @Update("""
            UPDATE sys_user
            SET status = #{status}, update_time = NOW()
            WHERE id = #{id} AND deleted = 0
            """)
    int updateStatus(@Param("id") Long id, @Param("status") Integer status);

    /** 统计某个角色下的用户数量（聚合查询示例） */
    @Select("SELECT COUNT(*) FROM sys_user WHERE role = #{role} AND deleted = 0")
    long countByRole(@Param("role") String role);
}
```

图 3.8　UserMapper.java（继承 BaseMapper 并补充注解 SQL）

这里记录一个实际踩到的坑。`ProjectMapper.selectByCategory` 最初写成普通的 `@Select`，查询返回的数据里 `name` 恒为 `null`。原因是：实体上的 `@TableField("project_name")`只对 MyBatis-Plus 自动生成的 SQL 生效，对开发者手写的注解 SQL 无效；手写 SQL 的结果集映射由 MyBatis 原生规则处理，开启驼峰映射后列 `project_name` 会被映射到并不存在的属性 `projectName`，于是 `name` 就一直是空。解决办法是用 `@Results`/`@Result` 显式声明列与属性的对应关系（或在 SQL

中直接取别名 project_name AS name）。修正后的写法如图 3.9 所示。

图 3.9 ProjectMapper.java（@Results 显式声明结果映射）（第 35–80 行）

```java
public interface ProjectMapper extends BaseMapper<Project> {

    /**
     * 按项目类别查询，用于演示注解 SQL 中的 WHERE 条件与排序。
     *
     * <p>{@code @Results} 中的 {@code id = "projectResultMap"} 给该映射关系命名，
     * 其它自定义查询可用 {@code @ResultMap("projectResultMap")} 复用，避免重复书写。
     */
    @Results(id = "projectResultMap", value = {
            @Result(column = "id", property = "id", id = true),
            @Result(column = "project_name", property = "name"),
            @Result(column = "category", property = "category"),
            @Result(column = "teacher", property = "teacher"),
            @Result(column = "deadline", property = "deadline"),
            @Result(column = "status", property = "status"),
            @Result(column = "description", property = "description"),
            @Result(column = "create_time", property = "createTime"),
            @Result(column = "update_time", property = "updateTime")
    })
    @Select("""
            SELECT id, project_name, category, teacher, deadline, status, description, create_time, update_time
            FROM training_project
            WHERE category = #{category}
            ORDER BY deadline
            """)
    List<Project> selectByCategory(@Param("category") String category);

    /** 按主键修改项目状态，返回受影响行数 */
    @Update("""
            UPDATE training_project
            SET status = #{status}, update_time = NOW()
            WHERE id = #{id}
            """)
    int updateStatus(@Param("id") Long id, @Param("status") Integer status);

    /**
     * 按类别分组统计项目数量。
     *
     * <p>返回值用 {@code Map} 承接「别名 → 列值」，适合结构不固定的聚合结果；
     * 每个 Map 的键即 SELECT 中的列别名（category、cnt）。
     */
    @Select("""
            SELECT category, COUNT(*) AS cnt
            FROM training_project
            GROUP BY category
            ORDER BY cnt DESC, category
```

图 3.9　ProjectMapper.java（@Results 显式声明结果映射）

（7）注册分页插件与字段自动填充处理器

Page 对象本身并不产生 LIMIT 语句，真正起作用的是分页插件 PaginationInnerInterceptor：它在 SQL 执行前拦截语句，改写成“SELECT

COUNT(*) 统计总数 + SELECT … LIMIT ?, ? 取当前页”两条物理分页语句。若不注册该插件，selectPage 会退化成查全表再在内存里截取。插件配置如图 3.10 所示。

图 3.10 MybatisPlusConfig.java（注册分页插件）（第 34–55 行）

```
34      public MybatisPlusInterceptor mybatisPlusInterceptor() {
35          MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();
36
37          // 分页插件：显式指定数据库类型为 MySQL，插件据此选择 LIMIT 语法方言
38          PaginationInnerInterceptor pagination = new PaginationInnerInterceptor(DbType.MYSQL);
39          // 单页最大条数限制：超过则按上限截断，防止 pageSize=999999 拖垮数据库
40          pagination.setMaxLimit(100L);
41          // 溢出处理：false 表示请求页码超出总页数时返回空列表，而不是回到第一页
42          pagination.setOverflow(false);
43
44          interceptor.addInnerInterceptor(pagination);
45          return interceptor;
46      }
47  }
```

图 3.10　MybatisPlusConfig.java（注册分页插件）

字段自动填充解决的是“创建时间、更新时间几乎每张表都有，却要在每个 Service 方法里重复赋值”的问题。实现 MetaObjectHandler 并配合实体上的 @TableField(fill = FieldFill.INSERT / INSERT_UPDATE)，插入与更新时 MyBatis-Plus 会自动回调填充，业务代码中不再出现 setCreateTime(...)。处理器实现如图 3.11 所示。

图 3.11 MyMetaObjectHandler.java（字段自动填充）（第 38–57 行）

```
38      public void insertFill(MetaObject metaObject) {
39          LocalDateTime now = LocalDateTime.now();
40          this.strictInsertFill(metaObject, "createTime", LocalDateTime.class, now);
41          this.strictInsertFill(metaObject, "updateTime", LocalDateTime.class, now);
42          log.debug("自动填充[插入] createTime/updateTime = {}", now);
43      }
44
45      /** 更新时只刷新更新时间，创建时间保持不变 */
46      @Override
47      public void updateFill(MetaObject metaObject) {
48          LocalDateTime now = LocalDateTime.now();
49          this.strictUpdateFill(metaObject, "updateTime", LocalDateTime.class, now);
50          log.debug("自动填充[更新] updateTime = {}", now);
51      }
52  }
```

图 3.11　MyMetaObjectHandler.java（字段自动填充）

（8）改造 Service 层：由内存 List 切换为 Mapper 访问数据库

Service 层的改造是本次实验工作量最集中的部分。以用户分页查询为例，实验二需要手工计算起始下标再在内存里 `skip/limit`；实验三只需构造 `Page` 与 `LambdaQueryWrapper` 两个对象，把条件用方法引用的形式交给 MyBatis-Plus 拼 SQL，分页由数据库完成。用 `LambdaQueryWrapper` 而非字符串列名的好处是：属性改名时编译期就会报错，不至于等到运行时才发现“列名写错”。分页方法的实现如图 3.12 所示。

图 3.12 UserServiceImpl.java 分页与条件构造（第 57–79 行）

```
57      public PageResult<User> page(int pageNum, int pageSize, String role, String keyword) {
58          // ① 分页对象：页码与每页条数，由分页插件翻译成 LIMIT 语句
59          Page<User> page = new Page<>(pageNum, pageSize);
60
61          // ② 条件构造器：所有条件都用「条件重载」写法，
62          //    第一个参数为 false 时该条件不会被拼进 SQL，因此无需在方法里写一堆 if
63          LambdaQueryWrapper<User> wrapper = new LambdaQueryWrapper<User>()
64                  .eq(StringUtils.hasText(role), User::getRole, role)
65                  // and(...) 把「账号模糊 or 姓名模糊」整体括起来，
66                  // 否则 or 会与前面的 role 条件平级，导致查询范围被意外放大
67                  .and(StringUtils.hasText(keyword), w -> w
68                          .like(User::getUsername, keyword)
69                          .or()
70                          .like(User::getRealName, keyword))
71                  .orderByDesc(User::getId);
72
73          userMapper.selectPage(page, wrapper);
74          log.info("分页查询用户：pageNum={}, pageSize={}, role={}, keyword={}, 命中 {} 条 / 共 {} 条",
75                  pageNum, pageSize, role, keyword, page.getRecords().size(), page.getTotal());
76          return new PageResult<>(page.getRecords(), page.getTotal(), pageNum, pageSize);
77      }
78
79      @Override
```

图 3.12　UserServiceImpl.java 分页与条件构造

查询条件全部采用“条件重载”写法，第一个参数为 `false` 时该条件不参与拼接，因此不必在方法里写一串 `if`。需要特别注意模糊查询的分组：`and(w -> w.like(...).or().like(...))` 必须把两个 `or` 条件整体括起来，否则 `or` 会与前面的 `role` 条件平级，把查询范围意外放大——这一点在服务端打印的 SQL 中可以直观看到括号的存在。

（9）实现用户与实训项目的增删改查接口

两个 Controller 各提供 8 个接口，覆盖 REST 风格中常用的动作与 HTTP 方法的对应关系，接口清单见表 3.8 与表 3.9。Controller 层不接触数据库，只负责“参数接收 → 调用 Service → 包装统一响应”，持久层细节全部下沉到 Service 与 Mapper。

表 3.8 用户模块接口清单（/api/users）

序号	HTTP 方法	路径	说明	涉及的 MyBatis-Plus 能力
1	GET	/page	分页 + 角色/关键字条件查询	selectPage + 分页插件
2	GET	/{id}	按主键查询详情	selectById
3	POST	/	新增用户	insert + 主键回填
4	PUT	/{id}	修改用户	updateById
5	DELETE	/{id}	删除用户（逻辑删除）	deleteById + @TableLogic
6	GET	/role/{role}	按角色查询	@Select 注解 SQL
7	PATCH	/{id}/status	启用 / 禁用用户	@Update 注解 SQL
8	GET	/statistics/role	按角色统计人数	@Select 聚合 SQL

表 3.9 实训项目模块接口清单（/api/projects）

序号	HTTP 方法	路径	说明	涉及的 MyBatis-Plus 能力
1	GET	/page	分页 + 状态/类别/关键字条件查询	selectPage + 分页插件
2	GET	/{id}	按主键查询详情	selectById
3	POST	/	新增实训项目	insert + 列名映射
4	PUT	/{id}	修改实训项目	updateById
5	DELETE	/{id}	删除实训项目（物理删除）	deleteById
6	GET	/category/{category}	按类别查询	@Select + @Results
7	PATCH	/{id}/status	修改项目状态	@Update 注解 SQL
8	GET	/statistics/category	按类别分组统计	@Select 聚合 SQL

（10）构建并启动应用，确认持久层装配生效

在工程根目录执行 `mvn -B clean package`，Maven 依次完成 clean、resources、compile、testCompile、test、jar、spring-boot:repackage 等阶段，输出 BUILD SUCCESS 并生成可执行 jar；随后执行 `java -jar target/ctts-0.0.1-SNAPSHOT.jar` 启动应用。启动日志如图 3.13 所示：HikariCP 连接池启动并连接 MySQL，MyBatis 以 `StdOutImpl` 作为日志实现初始化，内嵌 Tomcat 监听 8080 端口，最终输出 Started CttsApplication，说明数据源、持久层与 Web 容器的装配全部成功。

图 3.13 应用启动日志（数据源与 MyBatis-Plus 装配）

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.3.5)
19:31:00.158 INFO  [main] com.gzu.ctts.CttsApplication - Starting CttsApplication v0.0.1-SNAPSHOT using Java 17.0.
       20.1 with PID 26460 (C:\Users\asus\OneDrive\Desktop\javaweb\实验3\ctts\target\ctts-0.0.1-SNAPSHOT.jar started
       asus in C:\Users\asus\OneDrive\Desktop\javaweb\实验3\ctts)
19:31:00.159 DEBUG [main] com.gzu.ctts.CttsApplication - Running with Spring Boot v3.3.5, Spring v6.1.14
19:31:00.160 INFO  [main] com.gzu.ctts.CttsApplication - No active profile set, falling back to 1 default profile:
       "default"
19:31:02.215 INFO  [main] o.s.b.w.e.tomcat.TomcatWebServer - Tomcat initialized with port 8080 (http)
19:31:02.242 INFO  [main] o.a.catalina.core.StandardService - Starting service [Tomcat]
19:31:02.243 INFO  [main] o.a.catalina.core.StandardEngine - Starting Servlet engine: [Apache Tomcat/10.1.31]
19:31:02.318 INFO  [main] o.a.c.c.C.[Tomcat].[localhost].[/] - Initializing Spring embedded WebApplicationContext
19:31:02.319 INFO  [main] o.s.b.w.s.c.ServletWebServerApplicationContext - Root WebApplicationContext: initializat
       ion completed in 2098 ms
Logging initialized using 'class org.apache.ibatis.logging.stdout.StdOutImpl' adapter.
Get /192.168.1.238 network interface
Get network interface info: name:wlan2 (Intel(R) Wi-Fi 6E AX211 160MHz)
Initialization Sequence datacenterId:27 workerId:5
19:31:04.598 INFO  [main] o.s.b.w.e.tomcat.TomcatWebServer - Tomcat started on port 8080 (http) with context path
       '/'
19:31:04.616 INFO  [main] com.gzu.ctts.CttsApplication - Started CttsApplication in 4.996 seconds (process running
       for 5.657)
```

图 3.13　应用启动日志（数据源与 MyBatis-Plus 装配）

接口被调用后，服务端打印出 MyBatis-Plus 自动生成的 SQL，这是“持久层确实在工

作”最直接的证据，如图 3.14 所示。可以看到三件事：其一，分页查询被改写成 SELECT COUNT(*) 与带 LIMIT ? 的两条语句，属数据库侧物理分页；其二，条件构造器拼出的 WHERE 中，模糊查询的两个 or 条件被一对括号正确分组；其三，INSERT 语句的字段列表里自动带上了 create_time、update_time，这正是 @TableField(fill = ...) 与 MetaObjectHandler 配合的结果。

图 3.14 MyBatis-Plus 自动生成的 SQL（物理分页 / 条件构造 / 自动填充）

```
① 分页插件把「查全部」改写为「COUNT + LIMIT」两条物理分页语句

JDBC Connection [HikariProxyConnection@488600949 wrapping com.mysql.cj.jdbc.ConnectionImpl@2b433ff2] will not be m
        anaged by Spring
==>  Preparing: SELECT COUNT(*) AS total FROM sys_user WHERE deleted = 0 AND (role = ? AND (username LIKE ? OR rea
        l_name LIKE ?))
==> Parameters: STUDENT(String), %2024%(String), %2024%(String)
<==    Columns: total
<==        Row: 3
<==      Total: 1
==>  Preparing: SELECT id,username,real_name,role,email,phone,status,deleted,create_time,update_time FROM sys_user
        WHERE deleted=0 AND (role = ? AND (username LIKE ? OR real_name LIKE ?)) ORDER BY id DESC LIMIT ?
==> Parameters: STUDENT(String), %2024%(String), %2024%(String), 3(Long)
<==    Columns: id, username, real_name, role, email, phone, status, deleted, create_time, update_time

② 条件构造器拼出的 WHERE 与 ORDER BY (role = ? AND (username LIKE ? OR real_name LIKE ?))

<==      Total: 1
==>  Preparing: SELECT id,username,real_name,role,email,phone,status,deleted,create_time,update_time FROM sys_user
        WHERE deleted=0 AND (role = ? AND (username LIKE ? OR real_name LIKE ?)) ORDER BY id DESC LIMIT ?
==> Parameters: STUDENT(String), %2024%(String), %2024%(String), 3(Long)
<==    Columns: id, username, real_name, role, email, phone, status, deleted, create_time, update_time
<==        Row: 7, 2024003, 李娜, STUDENT, 2024003@qq.com, 13900000007, 0, 0, 2026-10-22 19:30:26, 2026-10-22 19:3
        0:26
<==        Row: 6, 2024002, 张伟, STUDENT, 2024002@qq.com, 13900000006, 1, 0, 2026-10-22 19:30:26, 2026-10-22 19:3
        0:26
<==        Row: 5, 2024001, 皮辰西, STUDENT, 2400770225@qq.com, 13900000005, 1, 0, 2026-10-22 19:30:26, 2026-10-22
        19:30:26

③ 插入语句: @TableField(fill) 让 create_time / update_time 自动进入字段列表

19:31:28.697 DEBUG [http-nio-8080-exec-3] c.g.ctts.config.MyMetaObjectHandler - 自动填充[插入] createTime/updateTim
        e = 2026-10-22T19:31:28.695403
==>  Preparing: INSERT INTO sys_user ( username, password, real_name, role, email, phone, status, deleted, create_
        time, update_time ) VALUES ( ?, ?, ?, ?, ?, ?, ?, ?, ?, ? )
==> Parameters: 2024008(String), e10adc3949ba59abbe56e057f20f883e(String), 赵敏(String), STUDENT(String), 2024008@
        qq.com(String), 13900000008(String), 1(Integer), 0(Integer), 2026-10-22T19:31:28.695403(LocalDateTime), 2026-
        22T19:31:28.695403(LocalDateTime)
<==    Updates: 1
Releasing transactional SqlSession [org.apache.ibatis.session.defaults.DefaultSqlSession@540781d]
19:31:28.707 INFO  [http-nio-8080-exec-3] c.g.c.service.impl.UserServiceImpl - 新增用户成功, id = 8, username = 2024
        008
Transaction synchronization committing SqlSession [org.apache.ibatis.session.defaults.DefaultSqlSession@540781d]
Transaction synchronization deregistering SqlSession [org.apache.ibatis.session.defaults.DefaultSqlSession@540781d
        ]
```

图 3.14　MyBatis-Plus 自动生成的 SQL（物理分页 / 条件构造 / 自动填充）

图 3.15 进一步给出两组对比证据。第一组是逻辑删除：删除用户时执行的其实是 `UPDATE sys_user SET update_time=?, deleted=1 WHERE id=? AND deleted=0`，数据行并未真正消失，而所有查询语句都被自动追加了 `deleted = 0`。第二组是列名映射：实体属性 `name` 对应列 `project_name` 时，MyBatis-Plus 自动生成了别名 `project_name AS name`；而手写的注解 SQL 不会享受这一待遇，必须靠 `@Results` 显式声明。这两组现象恰好解释了“为什么 BaseMapper 能直接用、自己写的 SQL 却要额外配置”。

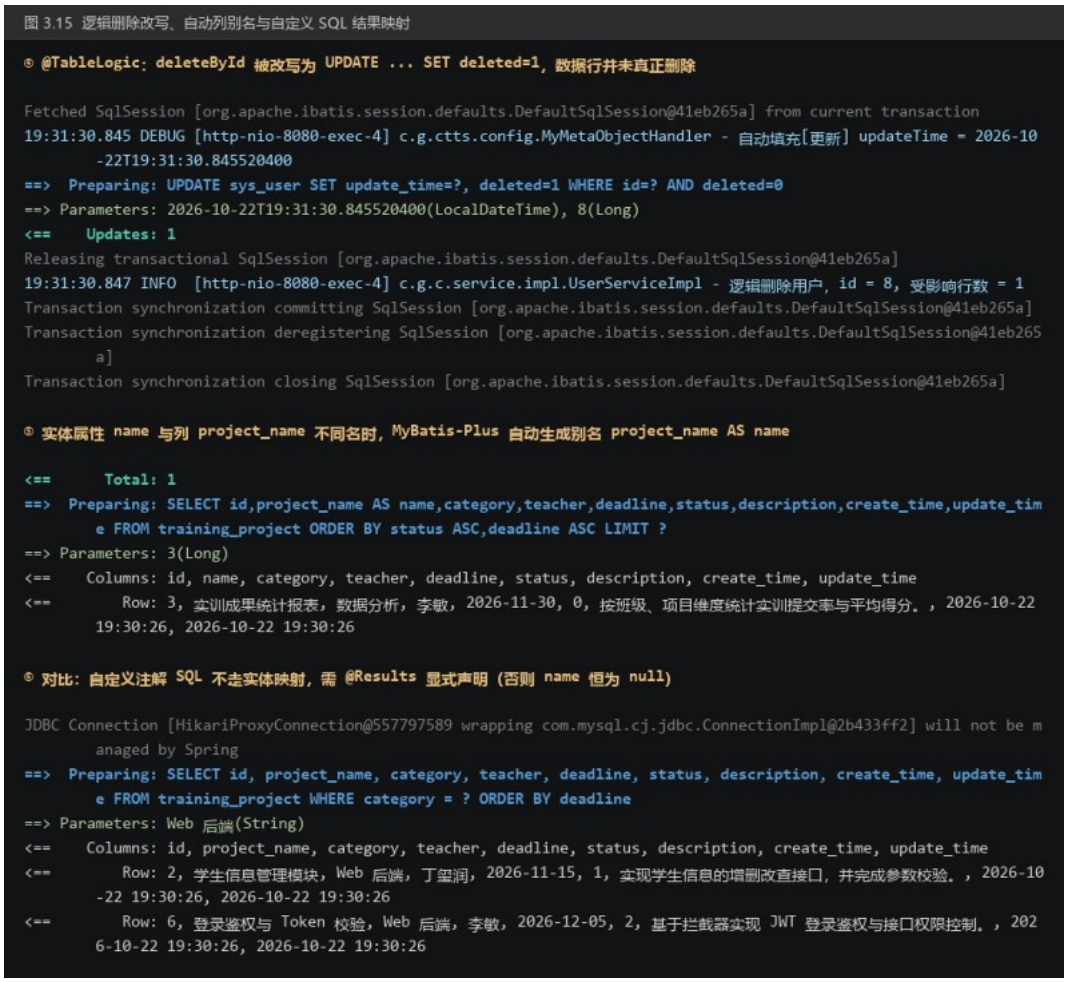


图 3.15　逻辑删除改写、自动列别名与自定义 SQL 结果映射

（11）使用 Apifox 验证全部接口

应用启动后访问 `http://localhost:8080/v3/api-docs` 即得到符合 OpenAPI 3 规范的接口文档，在 Apifox 中新建项目并以“URL 导入”方式填入该地址，即可把 16 个接口连同参数示例、响应结构一次性导入，无需逐个手工录入；导入后把前置 URL 设为 `http://localhost:8080` 便可直接发送请求。下面按模块给出各接口的真实请求与响应，请求体统一为 `Content-Type: application/json`。

① 用户分页查询与条件模糊查询（图 3.16）。分页接口返回 records、total、pages 等分页元信息；加上 role=STUDENT 与 keyword=2024 后，服务端 SQL 相应追加了角色等值与账号/姓名模糊匹配条件。

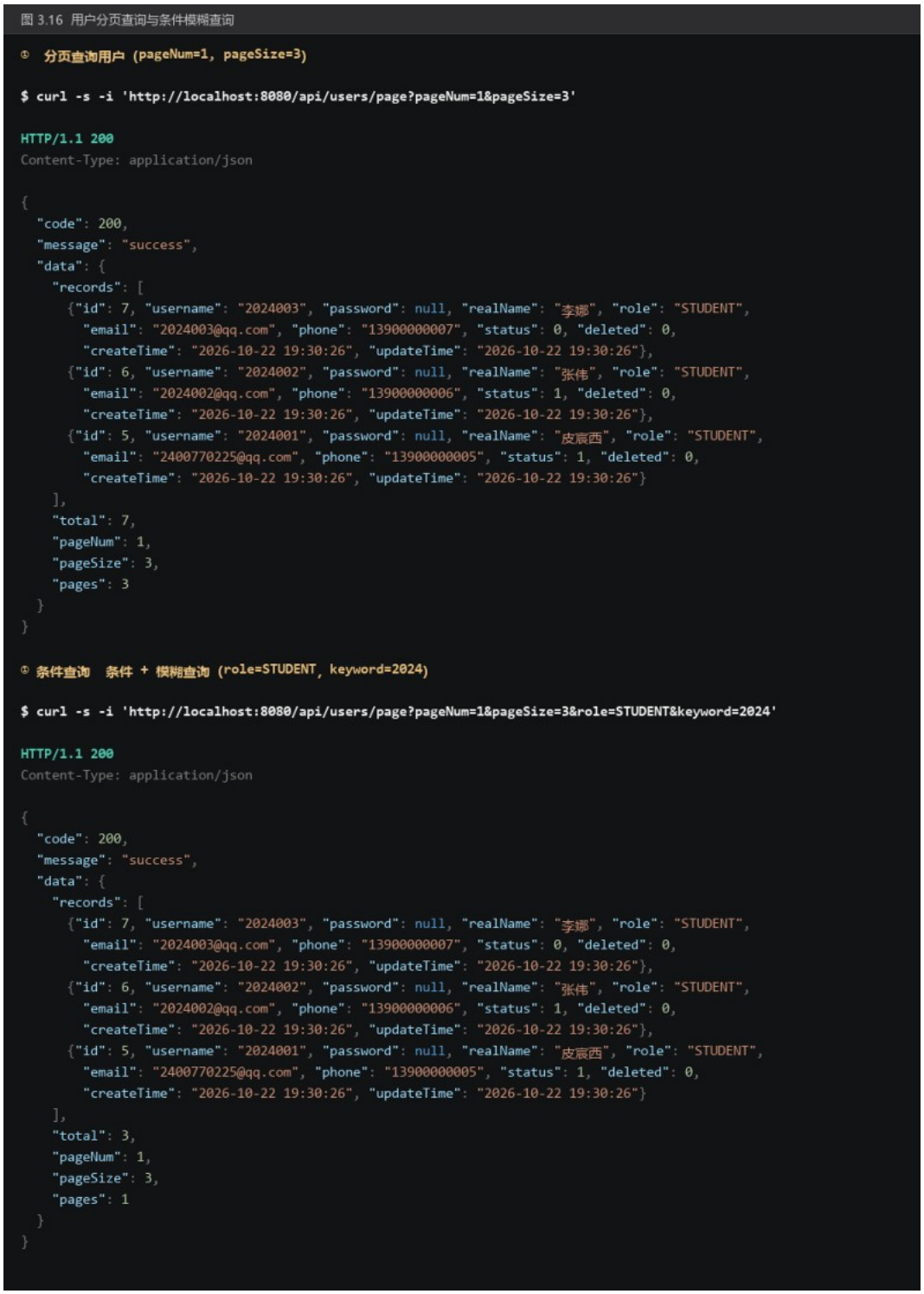


图 3.16　用户分页查询与条件模糊查询

② 按 id 查详情与异常分支（图 3.17）。password 字段因 @TableField(select = false) 在响应中恒为 null，密码摘要不会随接口泄露；id 不存在时返回

code=404；路径变量写成非数字（abc）时，由全局异常处理器归一为 code=400。

图 3.17 按 id 查用户详情（含 404 与类型不匹配）

```
② 按 id 查询用户详情（id=5），password 列因 select=false 不返回

$ curl -s -i 'http://localhost:8080/api/users/5'

HTTP/1.1 200
Content-Type: application/json

{
  "code": 200,
  "message": "success",
  "data": {
    "id": 5,
    "username": "2024001",
    "password": null,
    "realName": "皮宸西",
    "role": "STUDENT",
    "email": "2400770225@qq.com",
    "phone": "13900000005",
    "status": 1,
    "deleted": 0,
    "createTime": "2026-10-22 19:30:26",
    "updateTime": "2026-10-22 19:30:26"
  }
}

② 异常 id 不存在（999），Service 抛 BizException

$ curl -s -i 'http://localhost:8080/api/users/999'

HTTP/1.1 200
Content-Type: application/json

{
  "code": 404,
  "message": "用户不存在，id = 999",
  "data": null
}

② 异常 路径变量类型不匹配（id=abc）

$ curl -s -i 'http://localhost:8080/api/users/abc'

HTTP/1.1 200
Content-Type: application/json

{
  "code": 400,
  "message": "参数 id 类型不正确，期望 Long，实际收到：'abc'",
  "data": null
}
```

图 3.17　按 id 查用户详情（含 404 与类型不匹配）

③ 参数校验的两类失败（图 3.18）。pageSize=999 触发方法参数上的 @Max(50)；

新增用户时请求体中的账号过短、姓名为空、角色非法、邮箱格式错误则触发字段级校

验，多条错误信息用分号拼接后返回，前端可直接提示用户。

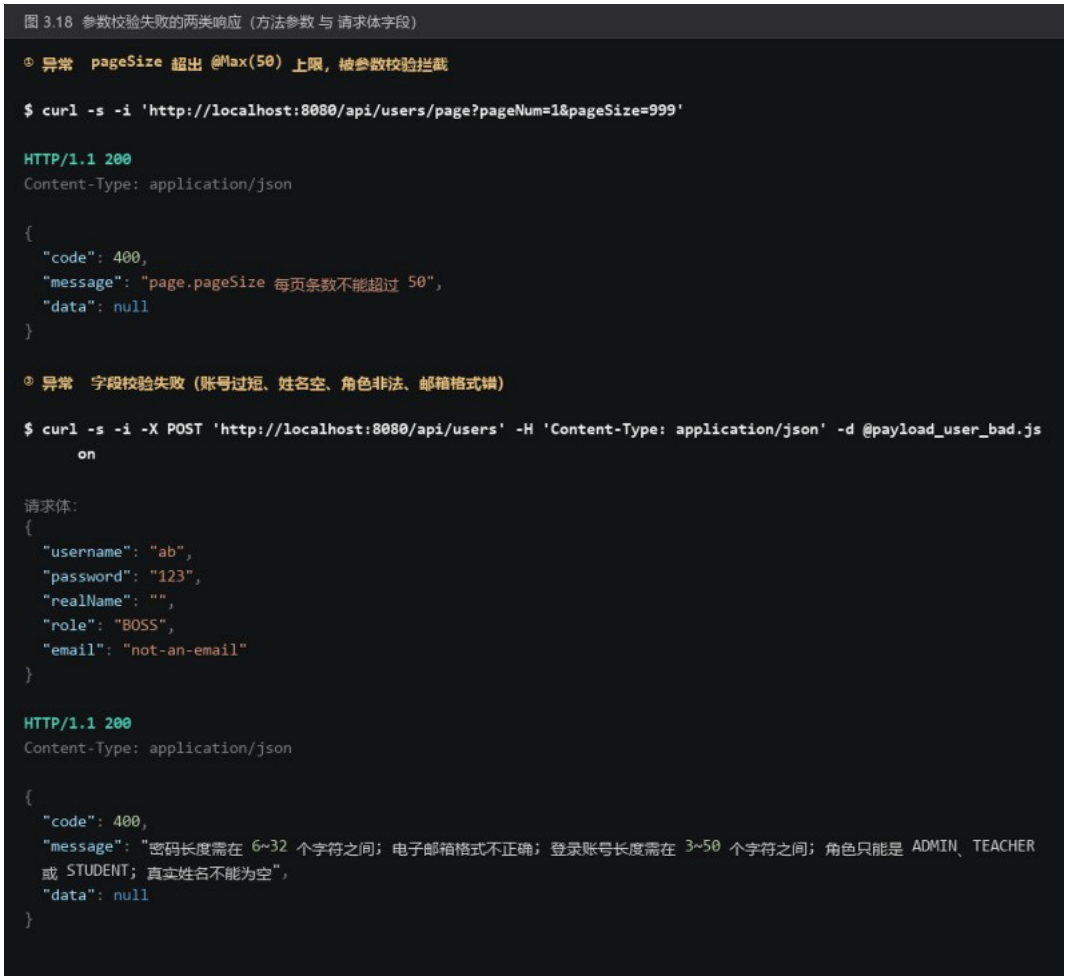


图 3.18　参数校验失败的两类响应（方法参数与请求体字段）

④ 新增用户与账号唯一性校验（图 3.19）。`BaseMapper.insert` 执行后，数据库生成的 6 号主键被自动回填到返回对象中；`createTime`、`updateTime` 由 `MetaObjectHandler` 自动填充。当账号 `admin` 已存在时，Service 层的前置校验返回 `code=400`；即便并发下漏过该校验，`sys_user.username` 上的唯一索引也会由数据库兜底。

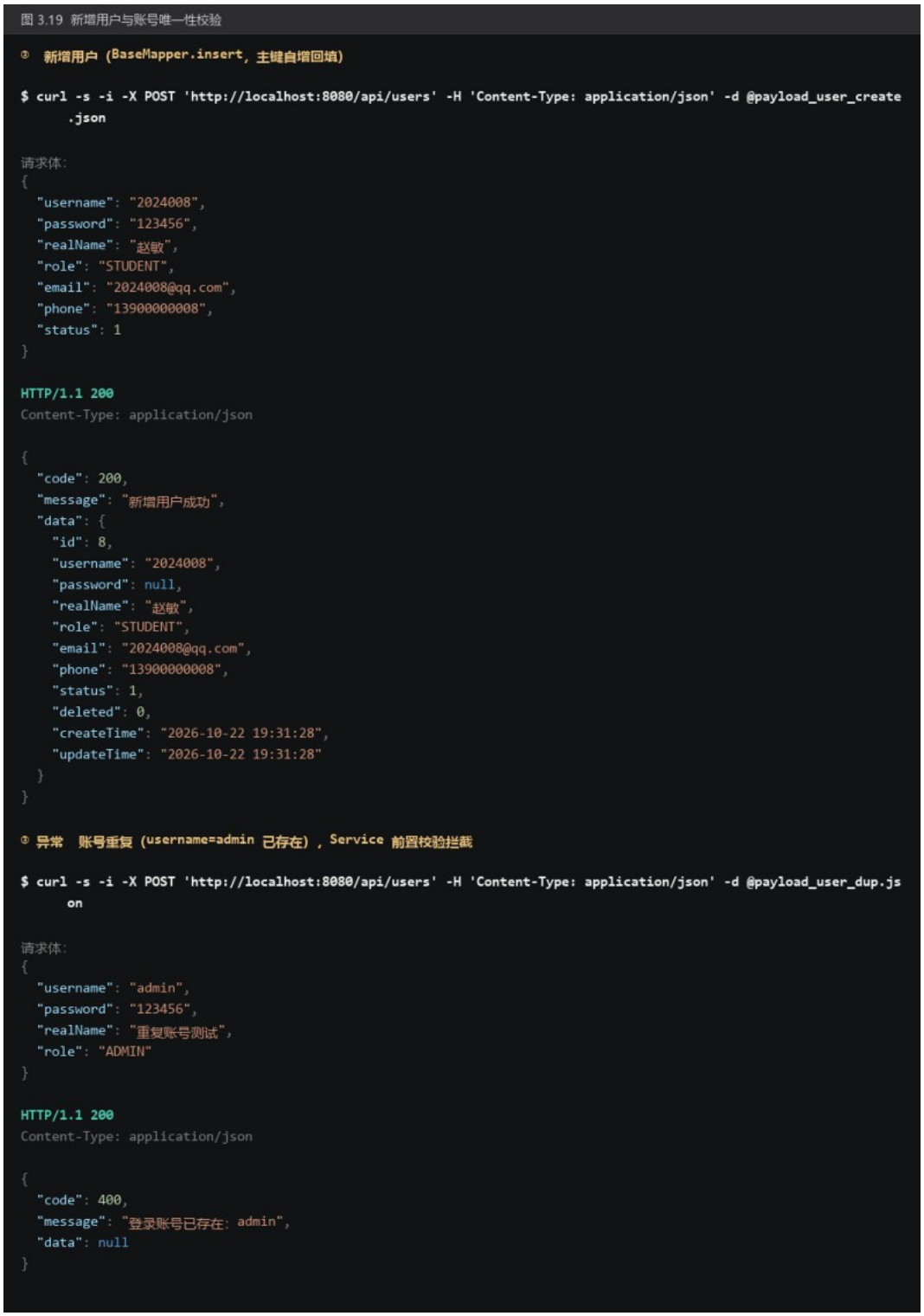


图 3.19　新增用户与账号唯一性校验

⑤ 修改用户与状态变更（图 3.20）。修改接口中 `password` 留空表示不修改原密码——MyBatis-Plus 默认的更新策略为 NOT_NULL，值为 `null` 的字段不会进入 `SET` 子

句，因此不会把密码覆盖成空；状态变更走的是自定义的 `@Update` 注解 SQL。

图 3.20 修改用户与启用/禁用状态（注解 SQL）

⑥ 修改用户（BaseMapper.updateById，password 留空表示不改密码）

```
$ curl -s -i -X PUT 'http://localhost:8080/api/users/8' -H 'Content-Type: application/json' -d @payload_user_updat
      e.json
```

请求体：

```
{
  "username": "2024008",
  "realName": "赵敏",
  "role": "TEACHER",
  "email": "zhaomin@ctts.edu.cn",
  "phone": "13900000009",
  "status": 1
}
```

```
HTTP/1.1 200
Content-Type: application/json
```

```
{
  "code": 200,
  "message": "修改用户成功",
  "data": {
    "id": 8,
    "username": "2024008",
    "password": null,
    "realName": "赵敏",
    "role": "TEACHER",
    "email": "zhaomin@ctts.edu.cn",
    "phone": "13900000009",
    "status": 1,
    "deleted": 0,
    "createTime": "2026-10-22 19:31:29",
    "updateTime": "2026-10-22 19:31:30"
  }
}
```

⑦ 禁用用户（自定义 @Update 注解 SQL，返回受影响行数）

```
$ curl -s -i -X PATCH 'http://localhost:8080/api/users/8/status?status=0'
```

```
HTTP/1.1 200
Content-Type: application/json
```

```
{
  "code": 200,
  "message": "修改用户状态成功",
  "data": {
    "id": 8,
    "username": "2024008",
    "password": null,
    "realName": "赵敏",
    "role": "TEACHER",
    "email": "zhaomin@ctts.edu.cn",
    "phone": "13900000009",
    "status": 0,
    "deleted": 0,
    "createTime": "2026-10-22 19:31:29",
    "updateTime": "2026-10-22 19:31:29"
  }
}
```

图 3.20　修改用户与启用/禁用状态（注解 SQL）

⑥ 按角色查询与按角色统计（图 3.21）。两个接口都走自定义注解 SQL，前者查询指定角色的用户列表，后者用 `COUNT(*)` 聚合出各角色人数。需要强调的是：自定义 SQL 不会自动追加逻辑删除条件，`deleted = 0` 需要在 SQL 中手写。

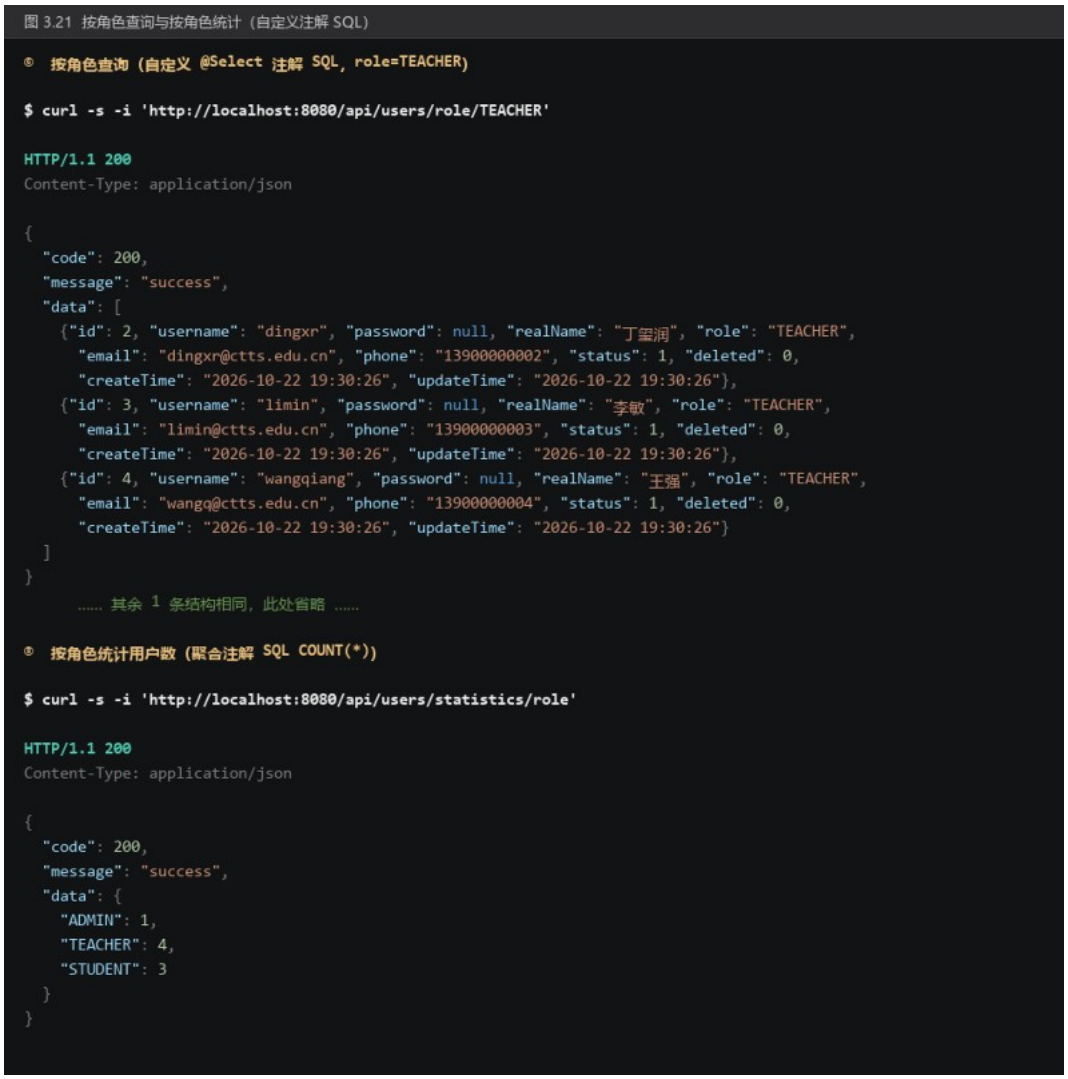


图 3.21　按角色查询与按角色统计（自定义注解 SQL）

⑦ 删除用户与逻辑删除验证（图 3.22）。调用删除接口后再次查询该 id，返回 `code=404`，但数据行仍在表中，只是 `deleted` 被置为 1。

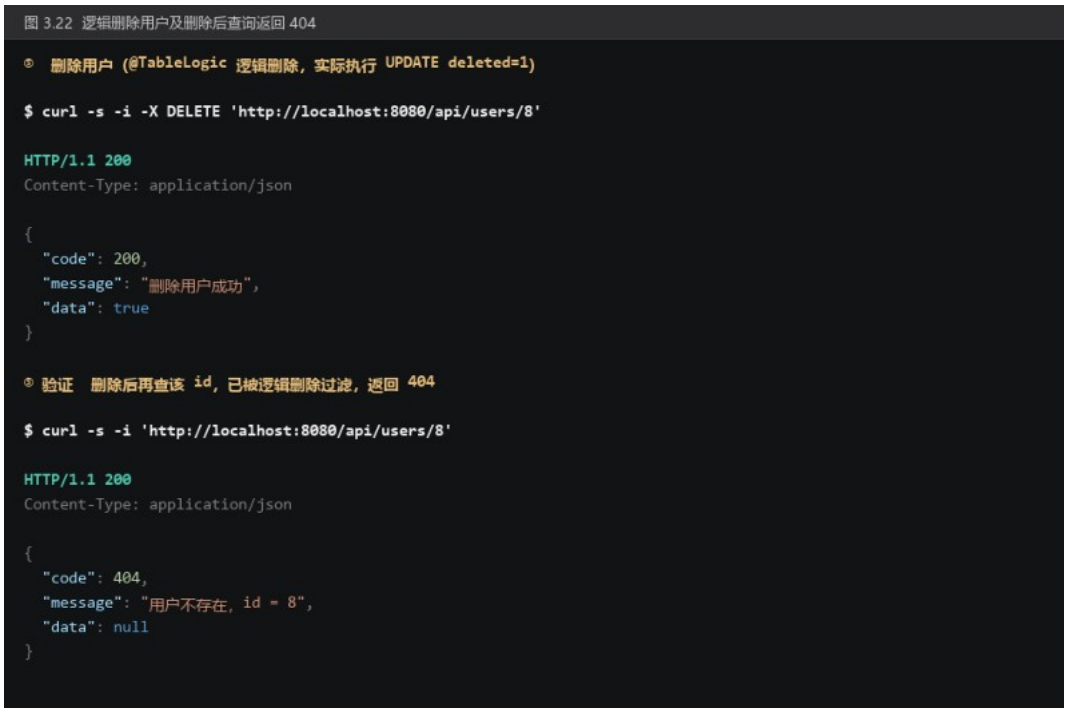


图 3.22 逻辑删除用户及删除后查询返回 404

⑧ 实训项目的分页与条件查询（图 3.23）。逻辑与用户模块一致，结果按“状态升序、截止日期升序”排列，便于按紧急程度查看。

图 3.23 实训项目分页查询与条件查询

① 分页查询实训项目 (pageNum=1, pageSize=3)

```
$ curl -s -i 'http://localhost:8080/api/projects/page?pageNum=1&pageSize=3'

HTTP/1.1 200
Content-Type: application/json

{
  "code": 200,
  "message": "success",
  "data": {
    "records": [
      {"id": 3, "name": "实训成果统计报表", "category": "数据分析", "teacher": "李敏", "deadline": "2026-11-30",
        "status": 0, "description": "按班级、项目维度统计实训提交率与平均得分。", "createTime": "2026-10-22 19:30:26",
        "updateTime": "2026-10-22 19:30:26"},
      {"id": 5, "name": "任务提交记录导出", "category": "工具开发", "teacher": "王强", "deadline": "2026-12-20",
        "status": 0, "description": "支持将学生提交记录导出为 Excel 文件。", "createTime": "2026-10-22 19:30:26",
        "updateTime": "2026-10-22 19:30:26"},
      {"id": 7, "name": "接口自动化测试用例", "category": "测试", "teacher": "王强", "deadline": "2026-12-25",
        "status": 0, "description": "为 Controller 层接口编写 JUnit 5 + MockMvc 自动化测试。",
        "createTime": "2026-10-22 19:30:26", "updateTime": "2026-10-22 19:30:26"}
    ],
    "total": 8,
    "pageNum": 1,
    "pageSize": 3,
    "pages": 3
  }
}
```

② 条件查询 条件查询 (status=1 且 category=Web 后端)

```
$ curl -s -i 'http://localhost:8080/api/projects/page?pageNum=1&pageSize=5&status=1&category=Web%20%E5%90%8E%E7%AB
    %AF'

HTTP/1.1 200
Content-Type: application/json

{
  "code": 200,
  "message": "success",
  "data": {
    "records": [
      {"id": 2, "name": "学生信息管理模块", "category": "Web 后端", "teacher": "丁玺润", "deadline": "2026-11-15",
        "status": 1, "description": "实现学生信息的增删改查接口，并完成参数校验。",
        "createTime": "2026-10-22 19:30:26", "updateTime": "2026-10-22 19:30:26"},
      {"id": 4, "name": "前后端分离接口联调", "category": "Web 后端", "teacher": "丁玺润", "deadline": "2026-12-10",
        "status": 1, "description": "使用 Apifox 对全部 REST 接口进行调试与文档生成。",
        "createTime": "2026-10-22 19:30:26", "updateTime": "2026-10-22 19:30:26"},
      {"id": 1, "name": "校园实训任务管理系统 (CTTS) 后端", "category": "Web 后端", "teacher": "丁玺润",
        "deadline": "2026-12-31", "status": 1, "description": "基于 Spring Boot 3 构建实训任务发布、提交与评分接口。",
        "createTime": "2026-10-22 19:30:26", "updateTime": "2026-10-22 19:30:26"}
    ],
    "total": 3,
    "pageNum": 1,
    "pageSize": 5,
    "pages": 1
  }
}
```

图 3.23 实训项目分页查询与条件查询

⑨ 按 id 查项目详情与不存在分支（图 3.24）。

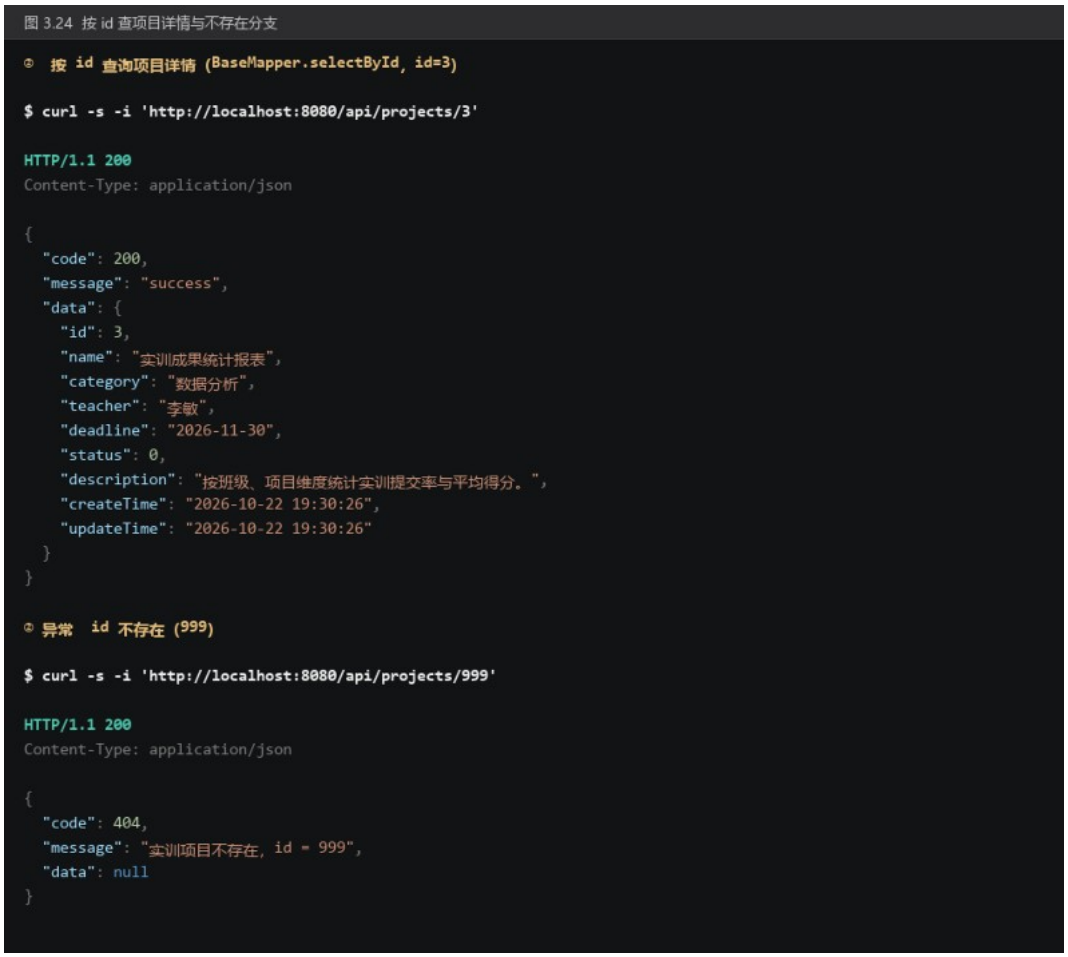


图 3.24　按 id 查项目详情与不存在分支

⑩ 新增实训项目与项目名校验（图 3.25）。该接口同时验证了中文、日期与列名映射：请求体中的 `deadline` 是 `yyyy-MM-dd` 字符串，经 Jackson 反序列化为 `LocalDate` 后写入 DATE 列；`name` 属性最终落库到 `project_name` 列。项目名为空时返回 `code=400`。

图 3.25 新增实训项目与项目名校验

```
⑩ 新增实训项目（含中文与日期，验证列名映射 project_name）

$ curl -s -i -X POST 'http://localhost:8080/api/projects' -H 'Content-Type: application/json' -d @payload_project_
      create.json

请求体：
{
  "name": "实训成果答辩系统",
  "category": "Web 后端",
  "teacher": "丁玺润",
  "deadline": "2027-01-08",
  "status": 1,
  "description": "支持答辩分组、评分与结果公示。"
}

HTTP/1.1 200
Content-Type: application/json

{
  "code": 200,
  "message": "新增实训项目成功",
  "data": {
    "id": 9,
    "name": "实训成果答辩系统",
    "category": "Web 后端",
    "teacher": "丁玺润",
    "deadline": "2027-01-08",
    "status": 1,
    "description": "支持答辩分组、评分与结果公示。",
    "createTime": "2026-10-22 19:31:32",
    "updateTime": "2026-10-22 19:31:32"
  }
}

⑩ 异常  项目名称为空，触发 @NotBlank 校验

$ curl -s -i -X POST 'http://localhost:8080/api/projects' -H 'Content-Type: application/json' -d @payload_project_
      badname.json

请求体：
{
  "name": "",
  "category": "Web 后端",
  "teacher": "丁玺润",
  "deadline": "2027-01-08"
}

HTTP/1.1 200
Content-Type: application/json

{
  "code": 400,
  "message": "项目名称不能为空",
  "data": null
}
```

图 3.25　新增实训项目与项目名校验

⑪ 日期格式校验失败（图 3.26）。把 `deadline` 写成 `2027/01/08` 时，Jackson 无法按 `yyyy-MM-dd` 解析，抛出 `HttpMessageNotReadableException`，由全局异常处

理器归一为 code=400 而不是 500——“参数格式错误”属于客户端错误，不应算作服务端故障。

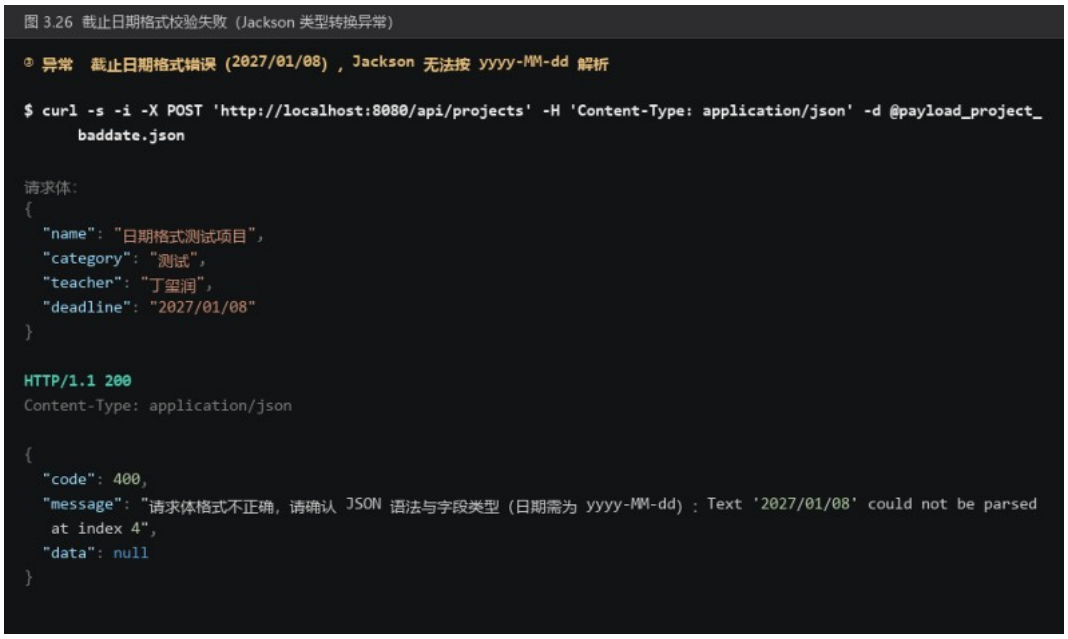


图 3.26 截止日期格式校验失败（Jackson 类型转换异常）

⑫ 修改实训项目与状态流转（图 3.27）。修改走 updateById，状态变更走自定义注解 SQL，并在返回结果中体现自动刷新后的 updateTime。

图 3.27 修改实训项目与状态流转（注解 SQL）

⑥ 修改实训项目（BaseMapper.updateById）

```
$ curl -s -i -X PUT 'http://localhost:8080/api/projects/9' -H 'Content-Type: application/json' -d @payload_project
      _update.json

请求体：
{
  "name": "实训成果答辩系统（V2）",
  "category": "Web 后端",
  "teacher": "李敏",
  "deadline": "2027-02-28",
  "status": 2,
  "description": "增加结果公示与成绩导出功能。"
}

HTTP/1.1 200
Content-Type: application/json

{
  "code": 200,
  "message": "修改实训项目成功",
  "data": {
    "id": 9,
    "name": "实训成果答辩系统（V2）",
    "category": "Web 后端",
    "teacher": "李敏",
    "deadline": "2027-02-28",
    "status": 2,
    "description": "增加结果公示与成绩导出功能。",
    "createTime": "2026-10-22 19:31:33",
    "updateTime": "2026-10-22 19:31:34"
  }
}
```

⑦ 修改项目状态为 2-已完成（自定义 @Update 注解 SQL）

```
$ curl -s -i -X PATCH 'http://localhost:8080/api/projects/3/status?status=2'

HTTP/1.1 200
Content-Type: application/json

{
  "code": 200,
  "message": "修改项目状态成功",
  "data": {
    "id": 3,
    "name": "实训成果统计报表",
    "category": "数据分析",
    "teacher": "李敏",
    "deadline": "2026-11-30",
    "status": 2,
    "description": "按班级、项目维度统计实训提交率与平均得分。",
    "createTime": "2026-10-22 19:30:26",
    "updateTime": "2026-10-22 19:31:33"
  }
}
```

图 3.27　修改实训项目与状态流转（注解 SQL）

⑬ 注解 SQL 的两个典型场景（图 3.28）。按类别查询验证了 @Results 结果映射修正后的效果，name 字段已能正确返回；分组统计则演示了 GROUP BY + COUNT(*) 这

类 BaseMapper 无法直接表达的查询。

图 3.28 按类别查询与分组聚合统计（注解 SQL）

⑤ 按类别查询（自定义 @Select 注解 SQL，category=Web 后端）

```
$ curl -s -i 'http://localhost:8080/api/projects/category/Web%20%E5%90%8E%E7%AB%AF'

HTTP/1.1 200
Content-Type: application/json

{
  "code": 200,
  "message": "success",
  "data": [
    {"id": 2, "name": "学生信息管理模块", "category": "Web 后端", "teacher": "丁望润", "deadline": "2026-11-15",
     "status": 1, "description": "实现学生信息的增删改查接口，并完成参数校验。", "createTime": "2026-10-22 19:30:26",
     "updateTime": "2026-10-22 19:30:26"},
    {"id": 6, "name": "登录鉴权与 Token 校验", "category": "Web 后端", "teacher": "李敏", "deadline": "2026-12-05",
     "status": 2, "description": "基于拦截器实现 JWT 登录鉴权与接口权限控制。", "createTime": "2026-10-22 19:30:26",
     "updateTime": "2026-10-22 19:30:26"},
    {"id": 4, "name": "前后端分离接口联调", "category": "Web 后端", "teacher": "丁望润", "deadline": "2026-12-10",
     "status": 1, "description": "使用 Apifox 对全部 REST 接口进行调试与文档生成。",
     "createTime": "2026-10-22 19:30:26", "updateTime": "2026-10-22 19:30:26"}
  ]
}
      …… 其余 2 条结构相同，此处省略 ……

⑥ 按类别聚合统计（GROUP BY category + COUNT(*)）

$ curl -s -i 'http://localhost:8080/api/projects/statistics/category'

HTTP/1.1 200
Content-Type: application/json

{
  "code": 200,
  "message": "success",
  "data": [
    {"cnt": 5, "category": "Web 后端"},
    {"cnt": 1, "category": "工具开发"},
    {"cnt": 1, "category": "性能优化"}
  ]
}
      …… 其余 2 条结构相同，此处省略 ……
```

图 3.28　按类别查询与分组聚合统计（注解 SQL）

⑭ 物理删除实训项目（图 3.29）。Project 未使用 @TableLogic，因此删除是真正的 DELETE FROM training_project WHERE id=?，删除后再次查询该 id 返回 code=404。与用户模块的逻辑删除形成对照。

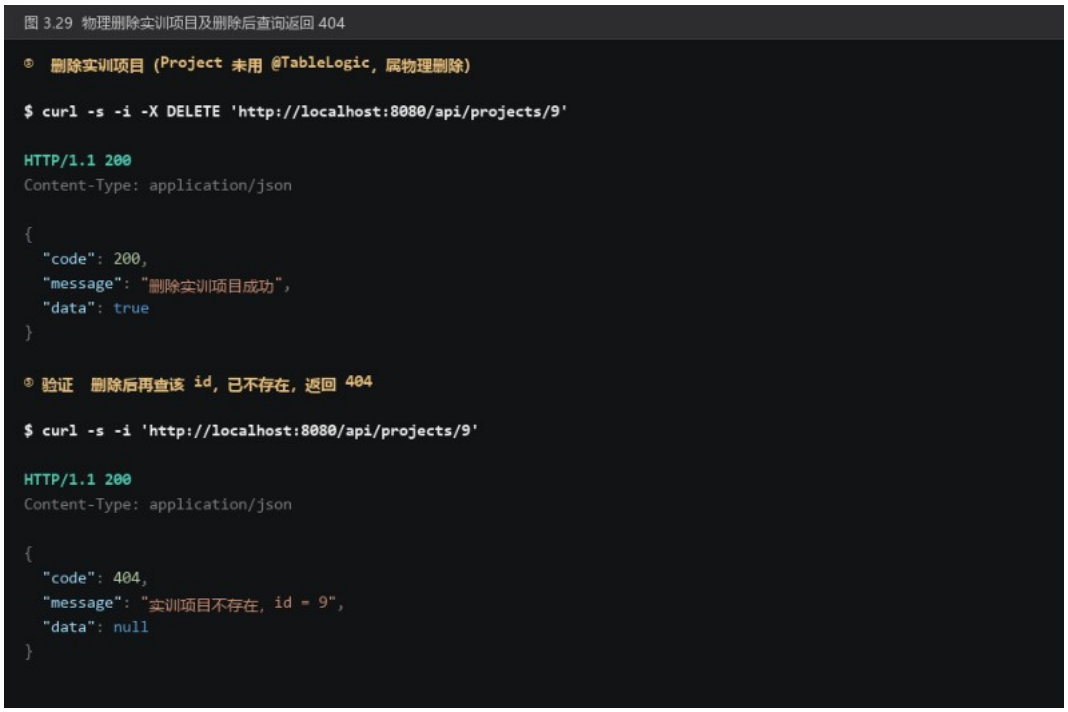


图 3.29　物理删除实训项目及删除后查询返回 404

全部接口验证完成后，直接查询数据库确认落库结果，如图 3.30 所示。被删除的用户（id＝8）数据行仍然存在，仅 `deleted` 为 1，故“有效用户数”为 7 而“全部用户数”为 8；被删除的实训项目则已从表中消失。至此，16 个接口全部验证通过，接口验证结果汇总见表 3.10。

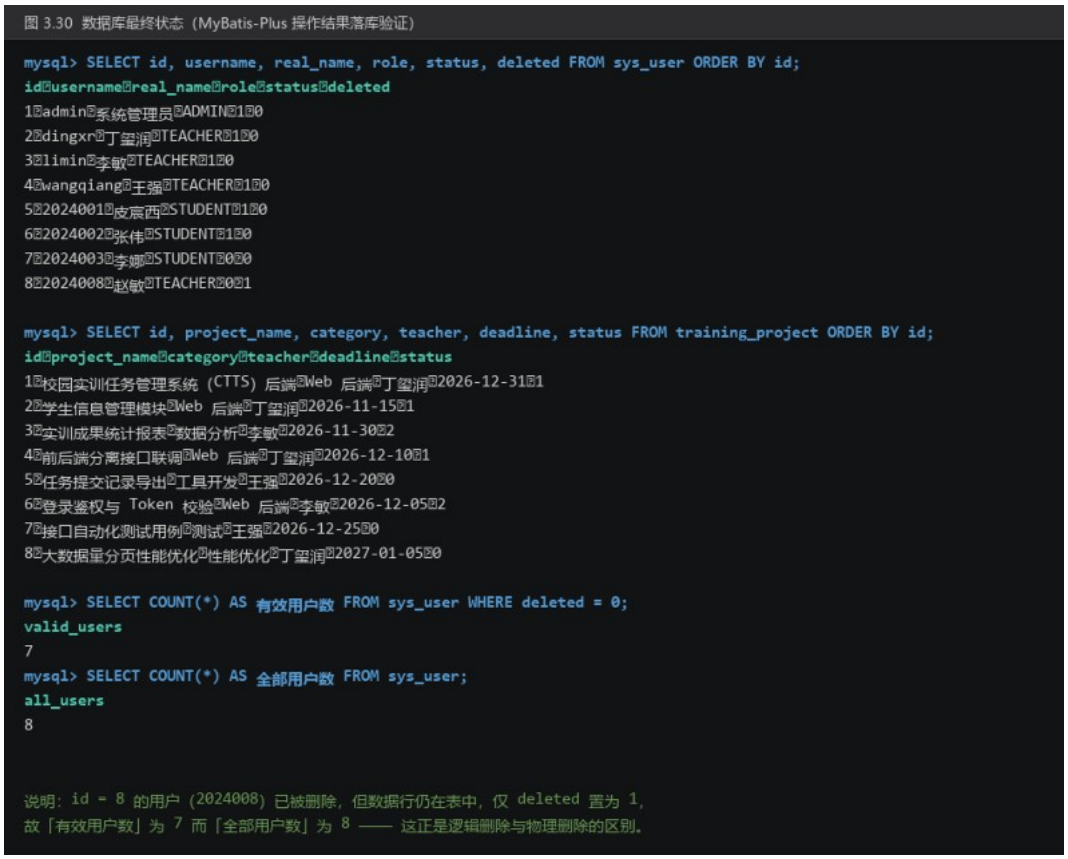


图 3.30　数据库最终状态（MyBatis-Plus 操作结果落库验证）

表 3.10 接口验证结果汇总

模块	接口数	验证通过的场景	结果
用户	8	分页、条件模糊查询、详情、404、类型不匹配、参数校验、新增与主键回填、账号重复、修改、状态变更、按角色查询、聚合统计、逻辑删除及验证	全部通过
实训项目	8	分页、条件查询、详情、404、新增、项目名校验、日期格式校验、修改、状态流转、按类别查询（注解 SQL）、分组统计、物理删除及验证	全部通过

五、实验总结：

1. 对 MyBatis-Plus 定位的理解。 实验二结束时，Service 层的数据还是内存里的一个 CopyOnWriteArrayList，进程一重启就全部还原。实验三把这条“内存数据库”换成了真实 MySQL，最直观的感受是：单表 CRUD 的 SQL 一行都没有写，插入、删除、按主键查询、分页查询全部由 BaseMapper 依据实体上的映射注解自动推导生成。MyBatis-Plus 的价值并不在于“取代 MyBatis”，而在于把“单表、条件由 Wrapper 表达”这一大类重复劳动自动化，使开发者可以把精力放在复杂查询与业务逻辑上；一旦遇到分组统计这类场景，仍然要回到 MyBatis 原生注解 SQL。

2. 对实体映射注解的理解。 实验前我以为 @TableName、@TableId、@TableField 只是“写全了好”，做完才发现它们是 BaseMapper 能工作的前提：框架在运行期正是靠这些注解推导表名、列名与主键策略。本次特意把实体属性取名为 name 而列名取为 project_name，制造了一处“不一致场景”。结果是 MyBatis-Plus 自动生成 SELECT id,project_name AS name,... 这样的别名，映射照常工作——这也从反面说明，注解缺失时框架并非“什么都不做”，而是按约定猜测，猜不到就会报 Unknown column 或者静

默返回 null。

3. 亲手踩到并解决的第一个坑：自定义注解 SQL 不走实体映射。

ProjectMapper.selectByCategory 最初只写了 @Select，返回结果里 name 恒为 null。排查后确认：@TableField("project_name") 只对 MyBatis-Plus 自动生成的 SQL 生效，手写 SQL 的结果集映射由 MyBatis 原生规则处理，开启驼峰映射后 project_name 会被映射到并不存在的属性 projectName。解决办法是用 @Results/@Result 显式声明列与属性的对应关系，或在 SQL 中直接取别名。这个坑让我真正分清了“框架生成的 SQL”与“我手写的 SQL”走的是两套映射机制。

4. 第二个坑：分页插件在 3.5.9 需要单独引入依赖。 首次执行 mvn clean package 时，因为只引入了 mybatis-plus-spring-boot3-starter，配置 PaginationInnerInterceptor 时找不到 SQL 解析器而失败。查阅后确认自 MyBatis-Plus 3.5.9 起，分页插件依赖的 jsqlparser 被拆分到独立的 mybatis-plus-jsqlparser 模块，必须单独声明。补上依赖后构建通过，控制台也印证了分页确实发生在数据库侧——SELECT COUNT(*) 统计总数，SELECT ... LIMIT ?,? 取当前页，而不是把全表读进内存再截取。

5. 对逻辑删除与字段自动填充的体会。 用户表上加了 @TableLogic 之后，deleteById

被自动改写成 UPDATE sys_user SET deleted=1 WHERE id=? AND deleted=0，所有查询语句也被自动追加 deleted = 0 条件：接口层面表现为“删掉了”，数据层面却仍可追溯，这正是真实业务中需要的效果。与之对照，实训项目表没有加该注解，删除就是真正的 DELETE，两者在数据库里的最终状态截然不同（有效用户数 7 而全部用户数 8）。字段自动填充则把创建/更新时间从每个 Service 方法中彻底消除，这类“每张表都有、每次都要写”的字段交给统一处理器显然更可靠。

6. 不足与下一步。 本次实验的接口尚未做登录鉴权与权限控制，所有接口都可被匿名调用，密码也只用 MD5 做了演示性摘要，真实项目应改用 BCrypt 等带盐慢哈希算法；此外事务目前只在单个 Service 方法内生效，跨表的一致性场景（如“新增项目的同时写入提交记录”）还没有验证。下一步计划补上登录鉴权与 Token 校验拦截器，并为 Mapper 与 Service 层编写 JUnit 5 + MockMvc 自动化测试，把本次靠手工点击验证的用例固化成可重复执行的测试。

六、教师评语：

评语教师：	丁圣润	实验评分：	